

this

CONTENT

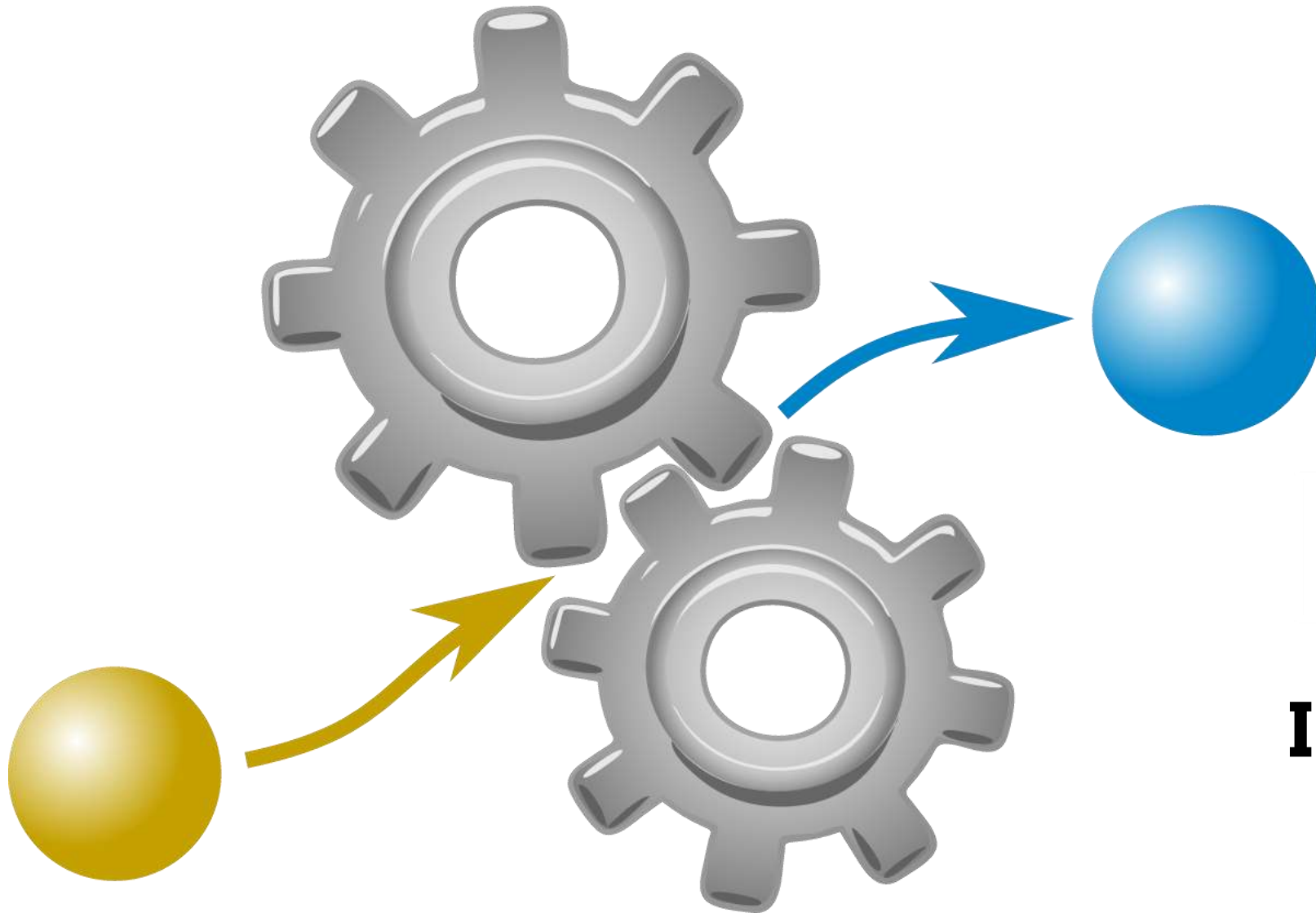
01 Introduction

02 Math Object

03 Number Methods

04 Date & Time





01

Introduction

Introduction

THIS

this refers to the object that is executing the current function.

The value of 'this' depends on how the function is called, not where it's defined.



this in Global Scope

The global scope is the outermost scope in JavaScript — outside of any functions, classes, or blocks.

- In a browser, the global object is window.
- In Node.js, it's global.

Value of this in Global Scope:

In Browser (non-strict mode):

```
console.log(this);           // ✓ window
```

In the browser, this in global scope refers to the window object — which is the global object.

```
var a = 10;
```

```
console.log(window.a);       // ✓ 10
```

```
console.log(this.a);         // ✓ 10 (because this === window)
```



this in Global Scope

In Browser (strict mode):

```
"use strict";
```

```
console.log(this);                // ✓ window (still global object)
```

In global scope, strict mode does not change the behavior of this. It still refers to the global object.

In Node.js:

```
console.log(this);                // ✓ {}
```

In Node.js, this in the global scope is an empty object ({}), not global.

⚠ But if you use this inside a function in the global scope, it behaves differently:

```
function test(){  
  console.log(this); // Node.js : global  
}
```

```
test();
```



Summary



Environment	this in Global Scope
Browser	window
Browser (strict mode)	window
Node.js (top level)	{ } (empty object)
Node.js (inside a function)	global object

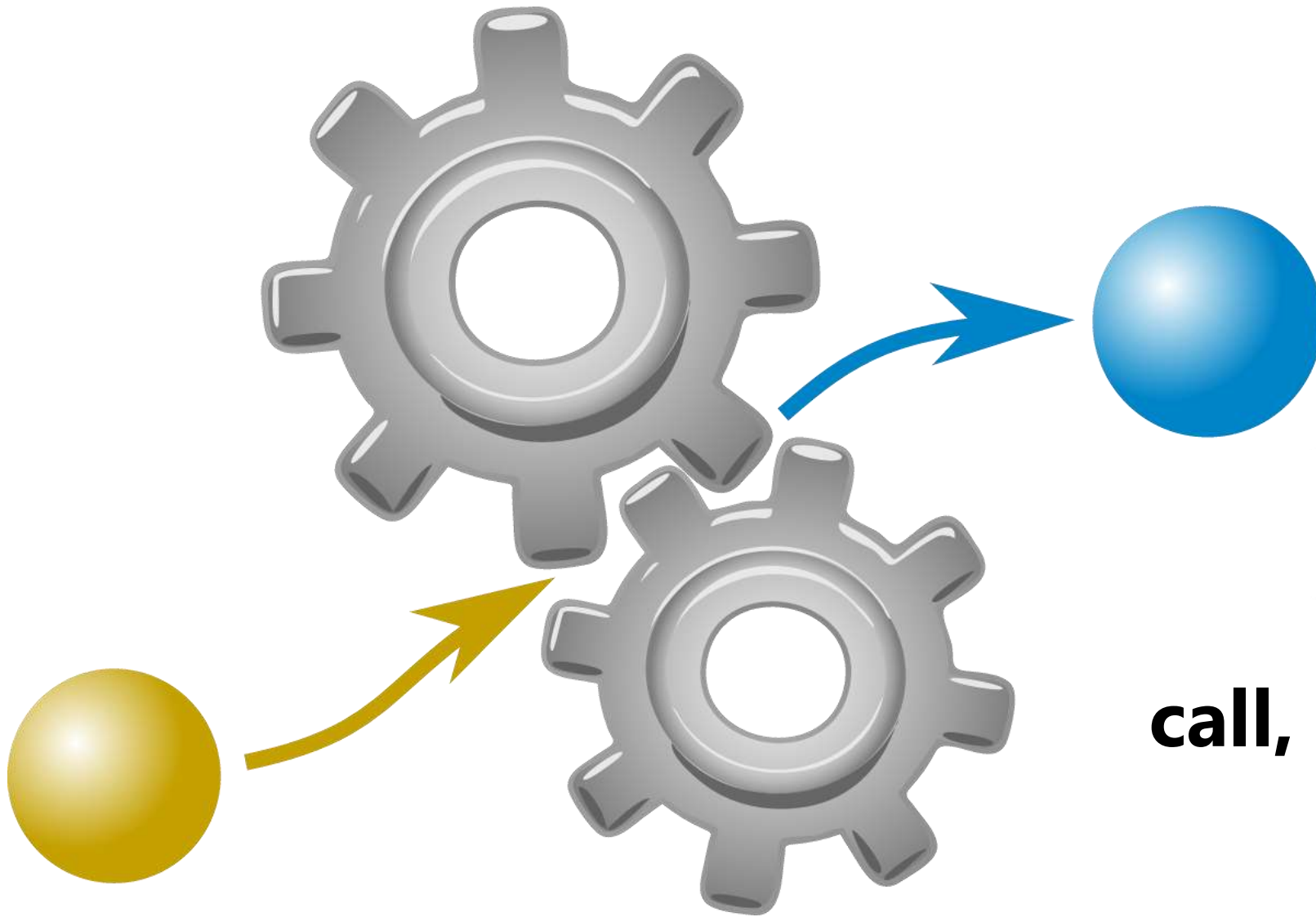
- **var declarations in global scope become properties of the global object (window in browsers).**
- **let and const do not attach to window**

```
var a = 1;
let b = 2;
```

```
console.log(window.a);
console.log(window.b);
```

```
// ✓ 1
// ✗ undefined
```



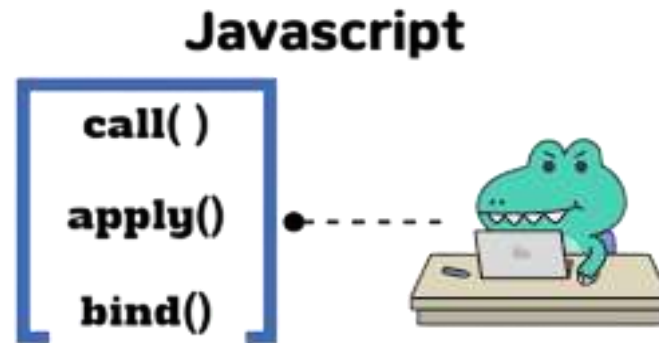


02

call, bind & apply

What are call(), apply(), and bind()?

They are methods available on JavaScript functions that let you manually control the value of this when calling a function.



Why They Matter :

- JavaScript functions are objects.
- You can borrow methods from one object for another.
- Control this dynamically.

